

sd15. Kafka-like Distributed Log

Bank #21 (◇ aggregator, platform-plausible; promoted with the distributed set). Feeds sd05's pipeline and #12's event stream; sd10 §14 has the mirror "queue vs log" defeater. JD tie-in: OCI streaming/telemetry backbone; per-partition ordering + ISR replication is the distributed-systems depth check.

Index

1. Crux & why hard
2. Clarifying questions
3. FR / NFR
4. Back-of-envelope
5. HLD
6. Storage: segments + sparse index (key code)
7. Replication: leader + ISR
8. Consumers: offsets, groups, rebalance
9. Data model / wire records
10. Delivery semantics
11. Failure modes
12. Trade-offs & defeaters
13. Pop-ups
14. Memorization card

1. Crux & why hard

- Naive: a DB table `messages(id serial, payload)` consumers poll → serial id = global write lock, deletes churn the table, each consumer re-scans, replay is a full-table nightmare.
- Minimum primitives: **partitioned append-only log** (ordering per partition, parallelism across), **sequential disk I/O** (append + page cache), **leader/ISR replication** (durability), **consumer-owned offsets** (replay for free).
- 🧠 "Ordering is *per partition*, never global — that one sentence prevents half the design mistakes."

2. Clarifying questions (assumed answers)

- Throughput/retention? → 500 MB/s cluster, 7 d retention, replay required
- Ordering scope? → per key (same key → same partition)
- Consumers? → many independent groups (pipeline, alerting, lake)
- Exactly-once? → at-least-once base; discuss idempotence/transactions as extension

3. FR / NFR

Req	Target	Enforced by
Append throughput	500 MB/s	sequential writes, batching, partitions
Durability	survive 1 broker loss	RF=3, acks=all, ISR
Per-key order	always	key → partition hash, single leader
Replay	any offset in retention	consumer seeks; log is immutable
Consumer scale	N consumers per group	partition = unit of parallelism

4. Back-of-envelope

- $500 \text{ MB/s} \div \sim 100 \text{ MB/s per-broker sequential} \approx 5 \text{ leaders} + \text{RF}=3 \rightarrow \sim 15 \text{ brokers with headroom}$ — so what: partition count (say 100+) must exceed max group size you'll ever want
- $7 \text{ d} \times 500 \text{ MB/s} \approx 300 \text{ TB} \times 3 \text{ replicas} \approx 900 \text{ TB}$ — so what: retention is a storage-cost decision, not a design constraint; tiered storage moves cold segments to object store (sd17)

5. HLD

```

producers —batch, key→partition→ brokers
                                topic T
                                ┌ p0: leader B1, ISR{B2,B3}
                                │ p1: leader B2, ISR{B1,B3}
                                └ p2: leader B3, ISR{B1,B2}
[hot path] append to leader → replicate to ISR → ack (acks=all)
consumers (group G): p0→c1, p1→c2, p2→c1  - pull, own their offsets
[control plane] controller (Raft/KRaft): partition→leader map, ISR membership

```

6. Storage: segments + sparse index (key code)

- Partition = chain of segment files (`00000000000000000000.log` , named by base offset) + sparse index (every ~4 KB: offset → file position). Same shape as sd04's logger, industrialized.

```

import bisect

class Segment:
    def __init__(self, base_offset):
        self.base = base_offset
        self.index = []          # [(rel_offset, file_pos)] sparse
        self.data = []          # records; file in real life
        self.next_rel = 0

    def append(self, batch_bytes):
        pos = sum(len(b) for b in self.data)
        if self.next_rel % 64 == 0:          # sparse: every 64 records
            self.index.append((self.next_rel, pos))
        self.data.append(batch_bytes)
        self.next_rel += 1
        return self.base + self.next_rel - 1    # absolute offset

    def find(self, abs_offset):
        rel = abs_offset - self.base
        i = bisect.bisect_right(self.index, (rel, float('inf'))) - 1
        idx_rel, pos = self.index[i]
        return idx_rel, pos          # scan forward from pos to rel
# sparse index: O(log) jump + short scan; index stays tiny and memory-resident

```

```

class Segment {
    final long baseOffset;
    private final NavigableMap<Long, Long> sparse = new TreeMap<>(); // relOff -> filePos
    private long nextRel = 0, filePos = 0;

    Segment(long base) { this.baseOffset = base; }

    long append(byte[] batch) {
        if (nextRel % 64 == 0) sparse.put(nextRel, filePos);
        filePos += batch.length;          // file channel write in real impl
        return baseOffset + nextRel++;
    }

    Map.Entry<Long, Long> find(long absOffset) {
        return sparse.floorEntry(absOffset - baseOffset); // then scan forward
    }
}
// floorEntry = the "closest indexed point before target" lookup

```

- Retention: delete whole sealed segments (cheap, no compaction); **compacted topics**: keep last record per key (changelog mode).

7. Replication: leader + ISR

- One leader per partition takes all reads/writes; followers fetch like consumers
- ISR = replicas caught up within lag bound; `acks=all` = leader waits for ISR quorum → $\approx W = \text{quorum}$ in `sd11` vocabulary
- Leader dies → controller elects new leader **from ISR** (no data loss); if ISR empty → unclean election = choose availability over consistency, off by default
- High-watermark = $\min(\text{ISR match offsets})$; consumers read only below it (no phantom reads of unreplicated data)

8. Consumers: offsets, groups, rebalance

- Pull model; each group owns `(group, partition) → committed offset` stored in an internal compacted topic
- Group of N consumers over P partitions: each partition exactly one consumer in the group; $N > P \rightarrow$ idle consumers
- Rebalance on join/leave/death (heartbeat timeout) → pause, reassign, resume from committed offsets; commit *after* processing = at-least-once

9. Data model / wire records

```
record batch: | base_off | len | crc | attrs(compression) | records[] |
record:       | rel_off | ts | key | value | headers |
offsets topic (compacted): key=(group,topic,partition) value=offset
controller state: topic → partitions → {leader, ISR[], epoch}
```

- Leader **epoch** fences a deposed leader's late writes — same fencing-token idea as `sd12`.

10. Delivery semantics

- At-least-once: commit after process (default story)
- At-most-once: commit before process (rarely wanted)
- Effectively-once: idempotent producer (seq per partition dedups broker-side) + idempotent consumer (`sd05 upsert`); full exactly-once = transactions (offsets + output in one atomic commit) — name it, don't build it in-round

11. Failure modes

- Broker down → leaders it held move to ISR replicas; producers/consumers re-fetch metadata and continue
- Slow follower → falls out of ISR (protects latency); rejoin after catch-up
- Consumer crash mid-batch → uncommitted offset → redelivery (at-least-once, downstream dedup)
- Hot partition (skewed key) → salt the key or split partitions; per-key order only within a salt

12. Trade-offs & defeaters

- Defeater: "consumers ack individual messages" — that's a work queue (SQS, sd10's model); a log tracks one offset per consumer, and replay/multi-subscriber is why you chose it
- Defeater: "global ordering" — one partition = no parallelism; per-key ordering is the honest contract
- Defeater: "push to consumers" — pull lets consumers control rate and enables replay/batching
- Queue vs log (say it): task fan-out once → queue; ordered replayable stream, many readers → log

13. Pop-ups

- ? Why is it fast on disk? → append-only sequential writes + page cache + zero-copy sendfile to consumers
- ? How do you size partitions? → target per-partition throughput (~10 MB/s) and max consumer parallelism; over-provision, they're cheap-ish
- ? Message too big? → claim check: payload to object storage (sd17), reference in the log
- **L5**: tiered storage (sealed segments → object store) decouples retention from broker disk; KRaft removes ZooKeeper

14. Memorization card

- Topic → partitions → segment chains + sparse index; ordering per partition via single leader
- RF=3, acks=all to ISR, high-watermark reads, leader-epoch fencing
- Consumers pull, own offsets (compacted topic), rebalance by group; commit-after = at-least-once
- Retention = drop segments; compaction = last-per-key; queue vs log = ack-per-msg vs offset